AI NOTES SERIES

Python & Scientific Computing

The Language Behind Modern Science and AI - Explained in Plain Words

Simple, clear notes for teachers, students, and curious beginners. What Python is, the basic ideas behind every program, the famous libraries scientists use, and how to start learning - with tiny examples you can try yourself.

By Shah Fahad

AI & Technology Innovator and Educator · Buner, Khyber Pakhtunkhwa, Pakistan

November 2026

GETTING STARTED

What Python Is

Python is a programming language - a way of writing clear instructions that a computer can follow, step by step.

A program is simply a set of instructions. Python is called a **general-purpose** language because the same language can be used for many different jobs: calculations, websites, small tools, data work, and artificial intelligence. It is not built for only one task.

Python is also known for being **readable**. Its way of writing, called its syntax, looks close to plain English. Lines are short, the structure is clear, and there is less extra punctuation than in many older languages. A beginner can often guess what a short Python program does just by reading it.

Why scientists and AI builders chose it

- **Clear syntax.** The code reads almost like sentences, so learners can focus on the idea instead of fighting with difficult symbols.
- **A huge library ecosystem.** Other people have already written ready-made collections of code, called libraries, for maths, tables, charts, and AI. You can use their work instead of starting from zero.
- **It is free.** Python itself is free to download and use, and so are its most famous libraries. A student with an ordinary computer can begin without paying anything.
- **A large, helpful community.** Because so many people use Python, answers to common problems are easy to find in books, courses, and discussion pages.

> **A simple way to remember it:** Python is like a clear, friendly workshop language. The language itself is simple to speak, and the workshop shelves are full of ready-made tools - the libraries - that other builders have left for you to use.

Scientific computing means using computers for science work: working with numbers, measurements, tables, and charts. Python became the common language for that work, and later for AI, because it made these jobs easier to do and easier to share.

CORE IDEAS

The Basic Ideas

Every Python program, large or small, is built from a few simple ideas. Learn these well and the rest becomes much easier.

The very first program most learners write simply shows a message on the screen:

```
print("Hello, world")
```

Variables - a named box for a value

A variable is a name that holds a value, like a labelled box. You put something in the box, and later you can use it by its name.

```
name = "Ahmad"
age = 12
```

Lists - a row of values

A list holds several values together in one row, in order. A class teacher might keep all the test marks in one list:

```
marks = [70, 85, 90]
```

From that one list, Python can work out useful answers. Adding all the marks and dividing by how many there are gives the average:

```
average = sum(marks) / len(marks)
print(average)
```

Here **sum** adds the values, **len** counts them, and the result printed is about 81.67.

CORE IDEAS

Repeating, Choosing, and Reusing

Loops - repeat an action

A loop repeats an action without you writing it again and again. This loop prints every mark in the list, one by one:

```
for mark in marks:
    print(mark)
```

Conditions - choose a path

A condition lets the program check something and choose what to do. The word **if** tests whether something is true:

```
if average >= 50:
    print("Pass")
else:
    print("Try again")
```

The spaces at the start of the inner lines matter in Python. They show which lines belong inside the loop or the condition. This is one reason Python code looks neat and tidy.

Functions - a reusable named recipe

A function is a small, named recipe: you write the steps once, give the recipe a name, and then use that name whenever you need those steps again.

```
def average(marks):
    return sum(marks) / len(marks)

print(average([70, 85, 90]))
```

> **Keep these five together:** variables hold values, lists hold rows of values, loops repeat actions, conditions choose a path, and functions package steps so you can reuse them.

THE TOOLS ON THE SHELF

The Famous Libraries

A library is a ready-made collection of code written by other people, which you can bring into your own program. These four names appear again and again in science and AI work.

NumPy

NumPy works with numbers and arrays - neat rows and grids of numbers - and does fast maths on all of them at once.

Pandas

Pandas works with tables, like a smart spreadsheet inside your program, and helps you clean, sort, and study data.

Matplotlib

Matplotlib draws charts - line graphs, bar charts, and other pictures that help you see patterns in your numbers.

scikit-learn

scikit-learn provides ready-made machine learning tools, so you can train simple AI models without building the maths from zero.

> **How they fit together:** a common pattern is to clean a table with Pandas, calculate with NumPy, draw the picture with Matplotlib, and - when the task calls for it - train a simple model with scikit-learn.

You do not need to master all four at the start. Learn plain Python first. Add one library when a real task needs it, and its purpose will make sense quickly.

PUTTING IT TO WORK

What You Can Do With It

- **Data cleaning and tables** - tidy messy lists and records, remove duplicates, and prepare neat tables for study.
- **Calculations** - totals, averages, percentages, and larger number work that would take a long time by hand.
- **Charts** - turn numbers into clear pictures that show change and comparison at a glance.
- **Automating repetitive file work** - rename, sort, or copy many files with one small program instead of doing each one by hand.
- **Connecting services through APIs** - let your program request information from another service and use the answer.
- **Building small tools** - make simple helpers for your own daily work, such as a marks calculator or a word counter.
- **Training simple AI models** - use libraries such as scikit-learn to teach a model from examples and test how well it learns.

Python in AI

Almost every AI course and AI library starts with Python, and there are practical reasons for that. The language is easy to read, so a learner can think about the AI idea itself - the data, the model, the result - instead of spending all their energy on difficult code.

The famous AI tools are also written to work with Python first, and course examples, books, and community answers are mostly in Python. Learning Python therefore opens the door to nearly every beginner-friendly AI resource.

HONEST AND PRACTICAL

Honest Limits

- **Speed has a cost.** Python is slower than compiled languages, such as C, when raw speed is all that matters. In practice, libraries like NumPy do the heavy number work in fast compiled code behind the scenes, so most science and AI work still runs well.
- **Code is only as correct as the person who checks it.** A program will happily calculate a wrong answer if the instructions or the data are wrong. The computer follows orders; checking the sense of the result is a human duty.
- **AI help still needs a human.** AI assistants can draft code quickly, but a draft is not proof. Running the code, reading what it does, and understanding it remain the human's job.

How to Start Learning

- **Install Python free from python.org.** The official website provides Python for common computer systems at no cost.
- **Practise small tasks daily.** Short, regular practice builds skill more reliably than rare, long sessions.
- **Read error messages calmly.** An error message is guidance, not failure. It usually names the line and the kind of problem; read it slowly and fix one thing at a time.
- **Build tiny real projects.** A marks average calculator, a simple quiz, or a file sorter for your own folders teaches more than reading alone.

> **An honest note from the writer:** these are study notes written from hands-on familiarity with Python and its tools. I am still learning myself, so treat this as a friendly starting map, not the final word.

REFERENCE

Words to Know

Keep this page as a quick reference. Every word here appears in the earlier chapters.

Word	What it means, in plain words
Program	A set of instructions, written in a language like Python, that tells a computer what to do, step by step.
Code	The written text of a program - the instructions themselves, as the programmer writes them.
Variable	A named place that holds a value, like a labelled box you can look inside or change later.
List	A row of values kept together in order under one name, such as a list of marks.
Loop	An instruction that repeats an action, so you write the steps once and the computer repeats them.
Function	A named, reusable recipe of steps. You write it once and call it by name whenever you need it.
Library	A ready-made collection of code written by others, which you can bring into your own program.
Bug	A mistake in code that makes a program behave wrongly or stop working.
Error message	The message Python shows when something goes wrong, naming where the problem is and what kind it is.
API	A set way for one program or service to talk to another - Application Programming Interface in full - like a fixed counter where requests are made and answers are given.

FINAL PAGE

Quick Revision

Read this box whenever you want the whole topic in one view.

The most important points

- **Python** is a readable, general-purpose programming language, and it is free to download and use.
- Scientists and AI builders chose it for its **clear syntax**, its **huge library ecosystem**, and its **large community**.
- The basic ideas are **variables** (a named box), **lists** (a row of values), **loops** (repeat), **conditions** (choose a path), and **functions** (a reusable recipe).
- The famous libraries are **NumPy** for numbers and arrays, **Pandas** for tables and cleaning, **Matplotlib** for charts, and **scikit-learn** for ready-made machine learning.
- With Python you can clean data, calculate, draw charts, automate file work, connect services through APIs, build small tools, and train simple AI models.
- Almost every AI course starts with Python because learners can focus on the ideas, not on fighting the language.
- Python is slower than compiled languages for raw speed, but its libraries do the heavy work in fast code behind the scenes.
- Code is only as correct as the person who checks it - and AI-drafted code still needs a human to run and understand it.
- Start by installing Python free from python.org, practise small tasks daily, read error messages calmly, and build tiny real projects.

A closing note

These notes are introductory study notes, and the explanations are simplified on purpose. Keep exploring each topic in depth, one small project at a time.

Thank you for reading. Start small, practise daily, and let each tiny working program give you the confidence for the next one.

Prepared by Shah Fahad

AI & Technology Innovator and Educator · Buner, Khyber Pakhtunkhwa, Pakistan